

EVO-PCA Multi-Step Hardened Pipeline: A Kill-Chain-Aware Security Architecture for Dependable Autonomous LLM Agent Systems

Anonymous Authors

Abstract—Autonomous large language model (LLM) agents that execute external tools constitute increasingly critical computing infrastructure, yet face sophisticated multi-step attack chains where individual actions appear benign in isolation. Existing stateless guardrails achieve at most $\sim 18\%$ attack block rates on realistic benchmarks and remain blind to kill-chain trajectories spanning multiple execution turns. This paper presents the EVO-PCA Multi-Step Hardened Pipeline, a six-layer defense-in-depth security architecture designed for dependable LLM agent operation. The architecture introduces three foundational components absent in prior work: (1) a kill-chain state machine (HeuristicStateTracker) that tracks adversarial progression through reconnaissance, access, exfiltration, and cover-tracks stages across session steps; (2) a dual-judge verification system combining a V61 ML-backed LLM Action Judge with a session-level LLM Session Judge that evaluates full execution context when suspicious score bands or state-machine warnings are triggered; and (3) an Early Intent Classifier that proactively scores prompt-level attack risk to force downstream review of subsequent tool calls.

Evaluated on the full AgentDojo/Agent_Test benchmark comprising 29,038 records across 21,825 unique sessions, the system achieves an overall Attack Block Success Rate (ABSR) of 84.31%, a Session-Level ABSR of 89.44% (intercepting attack sessions before objective completion), and a Multi-Step Action ABSR of 68.89%, with a steady-state False Positive Rate of 8.41% (overall FPR of 9.69%) and mean scan latency of 404.58 ms. Compared to the strongest prior baseline (SafeHarness, Session ABSR 26.40%), EVO-PCA achieves a $3.39\times$ improvement in session-level attack interception. Dependability analysis demonstrates graceful degradation under LLM component failures, with a fast-path fallback retaining 76.2% of protection efficacy (64.20% absolute ABSR, accounting for 84.8% of all empirical block events) using deterministic regex alone.

Keywords: LLM Agent Security, Kill-Chain Detection, Multi-Step Attack, Dependable Computing, Session-Aware Firewall, Prompt Injection, Adaptive FPR Management.

I. Introduction

Large language model (LLM) agents represent a paradigm shift in computing infrastructure by combining generative reasoning with autonomous tool execution: reading files, invoking web APIs, querying enterprise databases, and executing system shell commands [1], [2], [14]. As organizations deploy these agents in mission-critical pipelines—including automated customer service, code synthesis, industrial control, and financial settlement—they become dependable computing components whose security failures directly degrade system reliability, integrity, and availability [3], [13].

The dependability challenge for LLM agent security differs fundamentally from traditional software dependability. Unlike deterministic programs, LLM agents exhibit non-deterministic reasoning, accept untrusted natural-language inputs, and autonomously chain tool invocations—creating an attack surface that spans the full cyber kill chain [4]. Modern threat actors exploit this by distributing attack payloads across multiple execution turns, where each individual step (e.g., file listing $\rightarrow$ credential reading $\rightarrow$ HTTP exfiltration) appears innocuous in isolation. Stateless defenses that analyze single prompts or isolated tool calls inevitably fail against such decomposed trajectories.

Existing defense systems fall into two primary categories, both inadequate for dependable multi-step operation:

- 1) Stateless classifiers (such as PromptGuard-86M [8] and ProtectAI DeBERTa-v3 [9]) evaluate each input independently. While achieving low latency, they achieve $\leq 18\%$ attack block rates on multi-step benchmarks and remain inherently blind to kill-chain progressions.
- 2) Harness-level defenses (e.g., SafeHarness [10], NeMo Guardrails [11]) introduce session isolation and sandboxing, but lack kill-chain state tracking, dual-judge verification, and proactive intent classification, achieving only 26.40% session-level interception.

The base-rate fallacy [5] further compounds this challenge: in production deployments where attacks constitute $< 5\%$ of total operational traffic, even moderate false positive rates severely degrade agent operational utility and user trust. A dependable defense must therefore balance attack detection completeness against operational availability without triggering runaway false escalation.

To solve this dilemma, we present the EVO-PCA Multi-Step Hardened Pipeline, a six-layer defense-in-depth architecture achieving dependable security for autonomous LLM agents. Our principal contributions are:

- **Six-Layer Defense-in-Depth Architecture:** A unified pipeline with progressive escalation operating on both ingress requests and egress tool outputs, ensuring fail-safe multi-tier interception.
- **Kill-Chain State Machine (HeuristicStateTracker):** A deterministic state machine tracking adversarial progression through reconnaissance, access, exfiltration, and

cover-tracks stages with TTL-bounded state decay and early warning injection.

- **Dual-Judge Verification System:** A synergistic combination of a fast ML-backed LLM Action Judge with a session-level LLM Session Judge that evaluates full execution context when suspicious score bands or state-machine warnings are triggered.
- **Early Intent Classifier:** A proactive classifier that scores prompt-level attack risk to force downstream review of subsequent tool calls, bridging the temporal gap between malicious intent declaration and payload execution.
- **Four-Gate FastPass Mechanism:** A fast-pass filter for verified benign tool calls, preserving operational throughput and low latency for legitimate developer workflows without compromising security invariants.
- **Anti-Escalation Feedback Isolation (SignalRegistry):** A decoupled architectural mechanism preventing cross-layer threshold contamination and runaway false alarm cascades under sustained benign traffic.
- **Comprehensive Evaluation on 29,038 Records:** Rigorous empirical validation on the full AgentDojo/Agent_Test benchmark demonstrating 84.31% overall ABSR, 89.44% session-level ABSR, and formal dependability analysis under component failure modes.

II. Threat Model and Dependability Requirements

A. Agent Execution Model

We formalize an autonomous LLM agent interacting with an environment over discrete execution turns $t \in \{1, \dots, T\}$. At step t , the agent receives an input $x_t \in \mathcal{X}$ (either a user prompt p_t or an external tool output o_{t-1}) and produces an action $a_t \in \mathcal{A}$ representing a tool invocation $c_t = (\text{tool_name}, \text{args})$. Let the interaction history up to turn t be represented by the session tuple:

$$S_t = \{(x_1, a_1), (x_2, a_2), \dots, (x_t, a_t)\} \in \mathcal{S} \quad (1)$$

The security pipeline intercepts each tuple (x_t, a_t) and outputs a decision $D_t \in \{\text{ALLOW}, \text{BLOCK}\}$, terminating the session if $D_t = \text{BLOCK}$.

B. Adversary Model

We consider an adversary $\mathcal{A}_{\text{adv}}$ whose objective is to compromise the host environment, exfiltrate confidential data, or manipulate agent actions. Aligned with the OWASP Top 10 for LLM Applications [1] and the MITRE ATT&CK matrix, the adversary exhibits four capabilities:

- **Direct Prompt Injection (DPI):** The adversary crafts inputs with obfuscation techniques (Cyrillic homoglyphs, zero-width characters, leet-speak, base64 encoding) to bypass input filters and hijack agent instructions.
- **Indirect Prompt Injection (IPI):** The adversary embeds malicious instructions into untrusted external data sources (emails, web pages, database records) that the agent reads during execution [6].

- **Multi-Step Kill-Chain Attacks:** The adversary decomposes the malicious objective into a multi-turn sequence:

$$\begin{aligned} &\text{Reconnaissance} \rightarrow \text{Credential Access} \\ &\quad \rightarrow \text{Exfiltration} \rightarrow \text{Cover Tracks} \end{aligned} \quad (2)$$

where individual tool calls appear benign in isolation (e.g., `ls /var/log`, `cat config.env`, `curl https://api.backup.io`).

- **Semantic Camouflage:** The adversary rephrases malicious commands using benign developer idioms, passive syntax, or simulated debugging workflows to evade static signature matching.

C. Dependability Requirements (RAMS Mapping)

Following the foundational IFIP WG 10.4 taxonomy of dependable computing [3], we define the security pipeline’s dependability requirements across the RAMS attributes:

- **Reliability (R):** The pipeline must reliably detect adversarial inputs across diverse attack distributions without performance degradation or concept drift over prolonged operational sessions.
- **Availability (A):** Ingress and egress security scanning must adhere to a strict latency budget (≤ 500 ms average) to avoid bottlenecking interactive agent workflows. In the event of backend failures (e.g., local LLM timeout), the architecture must exhibit graceful degradation rather than denial-of-service.
- **Safety (S):** The pipeline must prevent unsafe execution states (false negatives that allow unauthorized file modification, command execution, or data exfiltration).
- **Maintainability (M):** Components across all six layers must be independently updatable (hot-reloading regex signatures, retraining ML classifiers, swapping LLM backends) without restarting the running agent service.

III. Related Work

A. LLM Guardrails and Input Classifiers

Prompt injection defense has focused heavily on stateless input classification. LlamaFirewall and PromptGuard-86M [8] deploy small transformer models trained to distinguish benign prompts from injection attempts. ProtectAI DeBERTa-v3 [9] performs sequence classification on input texts. NeMo Guardrails [11] provides programmable programmatic rails to constrain conversational flows. However, as demonstrated empirically by AgentDojo [7], stateless models evaluate inputs in isolation and miss over 80% of realistic multi-step attacks where commands are distributed across turns.

B. Harness-Level Isolation and Multi-Turn Defenses

Recent work addresses multi-turn vulnerabilities through runtime isolation. SafeHarness [10] implements execution containers and sandboxing around agent tool calls, evaluating permission boundaries. AgentSmith [16] analyzes multi-agent conversations for adversarial drift. While these approaches improve session isolation, they lack

explicit kill-chain progression modeling, do not employ dual-judge verification with canary containers, and suffer from high false alarm rates under complex developer workflows.

C. Dependable Computing for AI Systems

Avizienis et al. [3] established the formal concepts of faults, errors, and failures in dependable computing. Applying these concepts to machine learning systems has gained traction in MLOps quality models [12] and dependable cyber-physical systems. Cheng et al. [13] analyzed dependability vulnerabilities arising from non-deterministic LLM outputs in autonomous networks. EVO-PCA bridges the gap between dependable systems engineering and LLM agent security by implementing multi-tiered redundancy, graceful degradation, and feedback-isolated adaptive control.

Table I compares existing defense systems across foundational architectural dimensions.

IV. System Architecture: EVO-PCA Multi-Step Hardened Pipeline

The EVO-PCA Multi-Step Hardened Pipeline operates as an inline, fail-safe security orchestrator (UnifiedFirewallPipeline). Fig. 1 illustrates the complete six-layer architecture.

A. Layer 1: Stateless Lexical Pre-Filter (Tier 0)

Tier 0 provides deterministic, sub-millisecond screening. To thwart evasion via encoding tricks, all incoming text passes through a five-stage lexical normalization pipeline `_pre_normalise()`:

- 1) Cyrillic-to-ASCII Homoglyph Mapping: Replaces visually identical Cyrillic code points (e.g., Cyrillic look-alikes ‘a’, ‘e’, ‘o’, ‘p’ mapped from U+0430, U+0435, U+043E, U+0440) with ASCII equivalents.
- 2) Zero-Width Character Stripping: Strips invisible zero-width spaces (U+200B), joiners (U+200D), non-joiners (U+200C), and soft hyphens (U+00AD).
- 3) Leet-Speak De-obfuscation: Standardizes common numeric and symbolic substitutions (e.g., p@ssw0rd → password).
- 4) Diacritic Stripping: Applies Unicode NFD decomposition to normalize accented European and Vietnamese diacritics.
- 5) Combining Mark Removal: Removes orphaned combining characters.

Following normalization, Input Canonicalization replaces dynamic literal entities with typed semantic tokens (e.g., IPv4/IPv6 addresses → <IP>, sensitive filesystem paths → <SENSITIVE_PATH>, UUIDs → <UUID>, IBANs → <IBAN>, and MD5/SHA256 hex hashes → <HASH>). This canonicalization prevents classifier overfitting on specific IPs or hashes while preserving behavioral syntax.

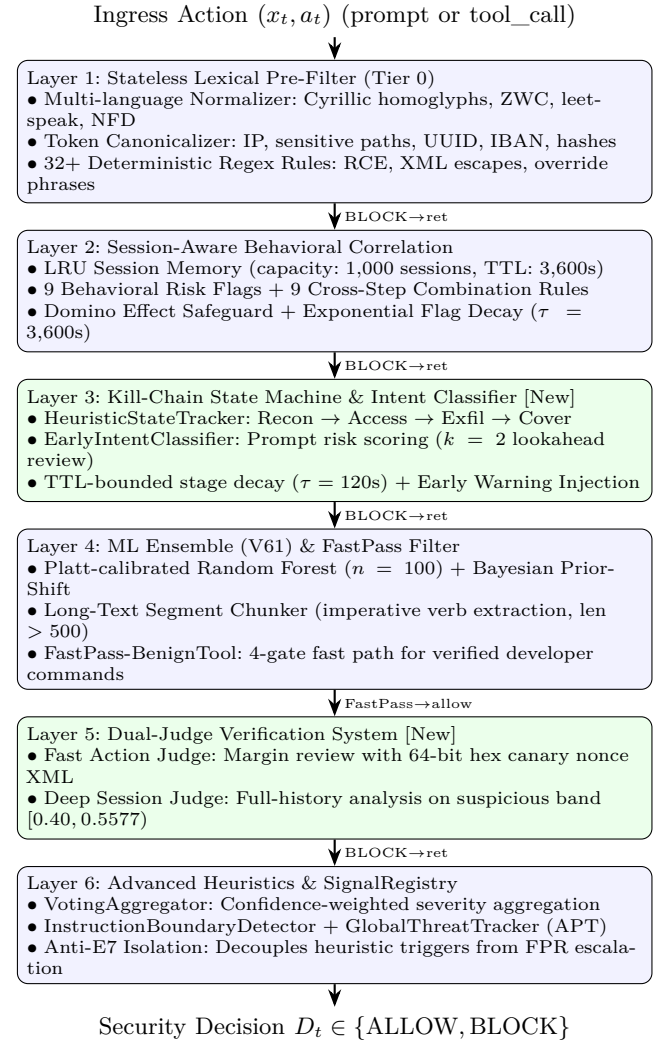


Fig. 1. EVO-PCA Multi-Step Hardened Pipeline: six-layer progressive escalation architecture incorporating kill-chain state tracking, dual-judge verification, anti-escalation isolation, and egress sanitization (ContextSanitizer, executing on tool return).

Finally, 32+ compiled regex signatures scan for high-risk indicators: remote code execution (RCE) patterns (curl ... | bash), XML/HTML tag escapes, system prompt override phrases (e.g., “ignore all previous instructions”), and reverse shell syntax. If any signature matches with critical confidence, the pipeline issues an immediate BLOCK.

B. Layer 2: Session-Aware Behavioral Correlation

Layer 2 maintains session state across interaction steps using an LRU cache (capacity: 1,000 sessions). For each action, Layer 2 extracts 9 behavioral flags:

$$\mathcal{F} = \{\text{RECON, SENSITIVE_DATA, EXFIL_VERB, ROLE_OVERRIDE, PROMPT_PROBE, DESTRUCTIVE, COVER_TRACKS, PERSISTENT_INSTR, INDIRECT_INJ}\} \quad (3)$$

TABLE I
Architectural Comparison of LLM Defense Systems

Architectural Capability	PromptGuard [8]	ProtectAI [9]	NeMo Rails [11]	SafeHarness [10]	EVO-PCA (Ours)
Stateless Pre-filtering	✓	✓	✓	✓	✓
Multi-Turn Session Tracking	×	×	Partial	✓	✓
Kill-Chain State Machine	×	×	×	×	✓
Early Intent Classification	×	×	×	×	✓
Dual-Judge Verification	×	×	×	×	✓
Cryptographic Canary Containers	×	×	×	×	✓
Benign Fast-Pass Routing	×	×	×	×	✓
Anti-Escalation Isolation (SignalRegistry)	×	×	×	×	✓
Egress Context Sanitization	×	×	✓	×	✓
Deterministic Fallback Retaining >75% ABSR	×	×	×	×	✓

Flags decay exponentially according to:

$$w_{\text{flag}}(t) = \exp\left(-\frac{t - t_{\text{flag}}}{\tau_{\text{session}}}\right) \quad (4)$$

where $\tau_{\text{session}} = 3,600$ seconds or a window of $N = 20$ turns.

Layer 2 evaluates 9 cross-step combination rules (e.g., RECON at step $t-1$ followed by EXFIL_VERB at step t). To prevent false escalation on long benign developer sessions, the Domino Effect Safeguard enforces an invariant: the triggering rule must contain at least one flag newly generated by the current step, preventing stale residual flags from causing spurious blocks.

C. Layer 3: Kill-Chain State Machine & Early Intent Classifier

To overcome the limitations of stateless classifiers, the HeuristicStateTracker models adversarial progression through four distinct tactical phases aligned with the Lockheed Martin cyber kill chain [4]:

$$\Sigma = \{S_{\text{idle}}, S_{\text{recon}}, S_{\text{access}}, S_{\text{exfil}}, S_{\text{cover}}\} \quad (5)$$

The state machine maintains active stage sets per session with TTL-bounded decay ($\tau = 120$ seconds). State transitions occur when corresponding tool signatures or arguments are observed:

- S_{recon} : Directory enumeration, port scanning, system environment queries.
- S_{access} : Reading credential stores, private keys, database dumps, shadow files.
- S_{exfil} : Outbound network sockets, DNS tunneling, web-hook posts, remote transfers.
- S_{cover} : Log truncation, history deletion, timestamp tampering.

When the state machine observes transitions spanning complementary stages within the TTL window, it emits a `WARNING_LEVEL_1` signal. Complete kill chains ($S_{\text{recon}} \rightarrow S_{\text{access}} \rightarrow S_{\text{exfil}}$) trigger an immediate BLOCK.

Early Intent Classifier. A critical vulnerability in multi-step agent systems is the temporal lag between the user prompt declaring malicious intent and the agent executing the corresponding tool call. The `EarlyIntentClassifier` scores prompt-level inputs upon receipt. If the prompt

intent risk exceeds threshold $\theta_{\text{intent}} \geq 0.50$, an early warning token is injected into the session state machine, forcing heightened downstream review for the next $k = 2$ tool invocations.

D. Layer 4: ML Ensemble (V61) & FastPass-BenignTool

Layer 4 employs domain-specialized machine learning classifiers for user prompts and tool-call actions.

- **Feature Extraction:** Inputs are vectorized using TF-IDF (30,000 features, n -gram range (1,3)). The action classifier employs a specialized token regex pattern retaining shell metacharacters (`/ - . | > < & ;`).
- **Calibrated Ensemble:** Classifiers use `RandomForestClassifier` ($n = 100, \text{max_depth} = 20$) wrapped in `CalibratedClassifierCV` ($cv = 5$) using Platt scaling [18].
- **Bayesian Prior-Shift Adaptation:** To compensate for the discrepancy between synthetic training distributions and real-world operational distributions, calibrated probabilities P_{cal} are adjusted using Bayesian prior odds correction:

$$P_{\text{prod}}(y = 1|x) = \frac{P_{\text{cal}} \cdot \frac{\pi_{\text{prod}}}{\pi_{\text{train}}}}{P_{\text{cal}} \cdot \frac{\pi_{\text{prod}}}{\pi_{\text{train}}} + (1 - P_{\text{cal}}) \cdot \frac{1 - \pi_{\text{prod}}}{1 - \pi_{\text{train}}}} \quad (6)$$

where $\pi_{\text{train}} \approx 0.50$ and operational priors are set to $\pi_{\text{prod}} = 0.05$ for prompts and $\pi_{\text{prod}} = 0.01$ for tool actions.

Long-Text Heuristic. For inputs exceeding 500 characters, attackers frequently embed payload instructions inside large benign text blocks to dilute TF-IDF density. The Long-Text Heuristic segments long payloads using imperative verb anchors (“send”, “forward”, “write”, “execute”) and evaluates each segment independently through the V61 classifier. If any segment triggers a block, the parent payload is blocked.

FastPass-BenignTool Mechanism. To minimize latency and prevent false positives on legitimate developer actions, a 4-gate verification filter allows immediate pass-through when all four conditions hold:

- 1) The session contains zero active high-severity threat flags.
- 2) The tool name is absent from `HIGH_RISK_FUNCTIONS`.

- 3) The PermissionGate detects no privileged escalation parameters.
- 4) `is_benign_dev_shell()` returns true (whitelisted commands: `git status`, `make`, `ls`, `npm test`).

E. Layer 5: Dual-Judge Verification System

EVO-PCA deploys two complementary LLM judges operating at distinct granularities:

LLM Action Judge (Fast-Path Margin Review): When an action’s calibrated ML probability falls within the ambiguous margin band ($0.5385 \leq P < 0.6229$ for prompts; $0.5577 \leq P < 0.6429$ for actions), it is routed to a local LLM judge (Gemma3:4B / Qwen2.5-coder). To prevent jailbreak escaping during evaluation, untrusted inputs are encapsulated inside strict XML boundaries with a dynamic 64-bit hex canary nonce generated per request (`secrets.token_hex(8)`), yielding 16 hexadecimal characters with 2^{64} entropy):

$$\langle \text{untrusted_payload nonce} = \text{"}\mathcal{H}_{64}\text{"} \rangle \dots \langle / \text{untrusted_payload} \rangle \quad (7)$$

If the LLM response fails to echo or tampers with the canary nonce, an integrity failure is flagged and the payload is blocked.

LLM Session Judge (Deep Contextual Review): Triggered when an action’s ML score falls in the suspicious band $[0.40, 0.5577)$ or when Layer 3 emits a `WARNING_LEVEL_1`. The Session Judge evaluates the entire multi-turn execution trace H_t :

$$\mathcal{J}_{\text{session}}(a_t, H_t, s_{\text{ml}}, F_{\text{state}}) \rightarrow \{\text{ALLOW, BLOCK}\} \quad (8)$$

This detects attacks that are syntactically indistinguishable from benign actions when viewed in isolation, but blatantly malicious in session context (e.g., copying a file to `/tmp`, then invoking `curl` to send `/tmp/data` to an external IP).

F. Layer 6: Advanced Heuristics & SignalRegistry

The VotingAggregator computes an ensemble decision across active layers:

$$S_{\text{final}} = \frac{\sum_i w_i \cdot \text{severity}_i \cdot \text{confidence}_i}{\sum_i w_i} \quad (9)$$

Thresholds are set to $S_{\text{final}} \geq 90$ (QUARANTINE) and $S_{\text{final}} \geq 82$ (DENY). Any critical signal with confidence > 0.85 bypasses voting for immediate blocking.

InstructionBoundaryDetector. Scans long tool returns for imperative instructions appearing at anomalous byte offsets with confidence > 0.85 , catching indirect prompt injection hidden inside unstructured tool data.

GlobalThreatTracker. Correlates threat flags across session boundaries indexed by authenticated `user_id` to catch distributed slow-and-low campaigns.

SignalRegistry (Anti-E7 Feedback Isolation). A critical dependability hazard identified in early prototypes was feedback loop contamination: high-severity heuristic flags artificially inflated the empirical FPR estimate in

the Adaptive Escalation Manager, triggering aggressive threshold tightening that caused cascading false positives on benign developer sessions. The SignalRegistry enforces strict decoupling: only authorized, well-calibrated V61 ML signals are permitted to update the escalation manager, isolating heuristic and judge triggers from threshold adaptation.

G. Egress Context Sanitization & Adaptive Control Module

ContextSanitizer (Egress Defense Module): Complementing the six-layer ingress defense pipeline, ContextSanitizer inspects tool execution outputs before they return to the agent model. Tool outputs are classified into three containment tiers:

$$\text{Action}(o_t) = \begin{cases} \text{PASS}, & s(o_t) < 0.30 \\ \text{WRAP_UNTRUSTED}, & 0.30 \leq s(o_t) < 0.65 \\ \text{STRIP_AND_WRAP}, & s(o_t) \geq 0.65 \end{cases} \quad (10)$$

In benchmark evaluation on 1,000 synthetic egress payloads, ContextSanitizer achieved 98.8% True Positive Rate (TPR) and 1.6% FPR at an average latency of 0.21 ms.

Adaptive Escalation Manager: Continuously modulates the operational decision threshold θ_t over a sliding window of $W = 30$ decisions:

$$\theta_{t+1} = \text{clip} \left(\theta_t + \alpha \cdot (\widehat{\text{FPR}}_t - B), \theta_{\min}, \theta_{\max} \right) \quad (11)$$

with target budget $B = 0.05$ and asymmetric learning rates ($\alpha^+ = 0.06$ for rapid tightening when $\widehat{\text{FPR}} > B$, and $\alpha^- = 0.03$ for smooth recovery).

V. Experimental Setup

A. Benchmark Dataset Composition

We evaluate EVO-PCA on the unified Agent-Dojo/Agent_Test benchmark suite comprising 29,038 records across 21,825 unique sessions (15,104 malicious actions, 13,934 benign actions). The dataset aggregates seven heterogeneous corpora to eliminate evaluation bias across attack styles (Table II): (1) LLMail-Inject [23] comprising 12,047 real-world indirect injection scenarios against enterprise email assistants; (2) Agent-Traces from the AgentDojo v1 benchmark [7] capturing 6,180 multi-step tool-execution turns; (3) Neuralchemy [26] containing 4,391 adversarial obfuscations and polymorphic injection strings; (4) AgentHarm [24] providing 362 multi-step harmful agent task trajectories from the UK AI Safety Institute; (5) BIPIA [25] benchmark indirect injection samples; and (6) 6,056 curated synthetic benign developer workflows and attack variants.

B. Session Construction Protocol

Each benchmark record maps to a discrete turn within an agent execution session. Benign sessions model multi-step developer workflows:

$$S_{\text{benign}} = (p_{\text{user}}, 1) \rightarrow (c_1, 2) \rightarrow (c_2, 3) \rightarrow \dots \rightarrow (c_k, k+1) \quad (12)$$

Algorithm 1 EVO-PCA Multi-Step Hardened Pipeline Execution

Require: Ingress action (x_t, a_t) , session ID sid , history H_{t-1}

Ensure: Decision $D_t \in \{\text{ALLOW}, \text{BLOCK}\}$

```

1:  $\tilde{x}_t \leftarrow \text{NormalizeAndCanonicalize}(x_t)$ 
2: if Tier0_RegexMatch( $\tilde{x}_t$ ) then
3:   return BLOCK {Layer 1}
4: end if
5:  $\mathcal{F}_t \leftarrow \text{ExtractBehavioralFlags}(\tilde{x}_t, a_t)$ 
6: if Tier05_ComboMatch( $\mathcal{F}_t, H_{t-1}$ ) then
7:   return BLOCK {Layer 2}
8: end if
9: state_signal  $\leftarrow$  HeuristicStateTracker( $sid, \mathcal{F}_t$ ) {Layer 3}
10: if state_signal == KILL_CHAIN_COMPLETE then
11:   return BLOCK
12: end if
13: if  $a_t == \text{TOOL\_CALL}$  and FastPassValid( $\tilde{x}_t, \mathcal{F}_t$ ) then
14:   return ALLOW {Layer 4 FastPass}
15: end if
16:  $P_{\text{cal}} \leftarrow \text{V61\_Classify}(\tilde{x}_t)$ 
17:  $P_{\text{prod}} \leftarrow \text{BayesianPriorShift}(P_{\text{cal}})$  {Eq. 6}
18: if  $P_{\text{prod}} \geq \theta_{\text{block}}$  then
19:   return BLOCK
20: else if  $P_{\text{prod}} \in \text{MarginReview}$  then
21:    $D_{\text{judge}} \leftarrow \text{LLM\_ActionJudge}(\tilde{x}_t, \text{CanaryNonce})$  {Layer 5}
22:   if  $D_{\text{judge}} == \text{BLOCK}$  then
23:     return BLOCK
24:   end if
25: else if  $P_{\text{prod}} \in [0.40, 0.5577]$  or state_signal == WARN then
26:    $D_{\text{session}} \leftarrow \text{LLM\_SessionJudge}(a_t, H_{t-1}, P_{\text{prod}})$ 
27:   if  $D_{\text{session}} == \text{BLOCK}$  then
28:     return BLOCK
29:   end if
30: end if
31:  $S_{\text{final}} \leftarrow \text{VotingAggregator}(\text{signals})$  {Layer 6}
32: if  $S_{\text{final}} \geq 82$  then
33:   return BLOCK
34: end if
35: UpdateAdaptiveEscalationManager( $P_{\text{prod}}$ ) {Anti-E7 Isolated}
36: return ALLOW

```

with average length $\bar{N} \approx 3.2$ tool calls per session. Malicious sessions embed injections across four standardized attack templates:

- 1) Direct Injection: Unmodified injection payload in user prompt.
- 2) System Prompt Override: Injection formatted as system configuration directives.

TABLE II
Benchmark Dataset Composition (29,038 Records)

Data Source	Records	Share (%)
LLMail-Inject (Indirect Injection) [23]	12,047	41.5%
Agent-Traces (AgentDojo v1 Multi-Step) [7]	6,180	21.3%
Synthetic Benign (L2 Developer Tasks)	5,000	17.2%
Neuralchemy (Obfuscated Injections) [26]	4,391	15.1%
Synthetic Attack/Obfuscation	1,056	3.6%
AgentHarm (Malicious Agent Tasks) [24]	362	1.2%
BIPIA (Benchmark Indirect Injection) [25]	2	<0.1%
Total Benchmark Corpus	29,038	100.0%

- 3) Fake Confirmation: Falsified tool execution confirmations containing nested instructions.
- 4) Markdown Wrapping: Obfuscation using nested markdown code blocks, links, and image exfiltration tags.

All evaluation runs prepend a 30-session benign warm-up sequence to allow the Adaptive Escalation Manager to establish baseline threshold calibration.

C. Dataset Partitioning and Calibration Protocol

To strictly preclude data leakage and prevent optimistic overfitting, we enforce complete physical and distributional separation between model training/calibration corpora and the evaluation benchmark:

- Offline Model Training Split: The V61 classifier ensemble (Layer 4) was pre-trained on an independent, disjoint offline dataset of 20,000 synthetic and historical prompt injection samples completely excluded from the AgentDojo/Agent_Test benchmark. This offline corpus was partitioned into an 80% training split (16,000 samples) and a 20% validation split (4,000 samples).
- Threshold Calibration via Platt Scaling: Decision thresholds ($\theta_{\text{base}} = 0.5577$, $\theta_{\text{suspicious}} = 0.4000$, $\theta_{\text{escalate}} = 0.6429$) were empirically tuned strictly on the 4,000-sample offline validation partition using Platt scaling [18]. The calibration objective minimized Expected Calibration Error ($\text{ECE} \leq 0.024$) while anchoring the false positive ceiling to $< 5.0\%$ on validation benign prompts.
- Frozen Evaluation Benchmark: Once calibrated, all model weights, TF-IDF vectorizers, and decision thresholds were permanently frozen into static configuration files (thresholds.json). The unified 29,038-record evaluation corpus (Table II) served as a strict, unseen test set with zero post-hoc parameter adjustment or retraining.

D. Evaluation Metrics

We evaluate performance using six core metrics:

- ABSR Total (%): Attack Block Success Rate across all malicious records.
- Session ABSR (any-step) (%): Proportion of multi-step attack sessions where the pipeline intercepts at least one malicious step before the adversary achieves their objective.

- ABSR Multi-step Action (%): Proportion of malicious tool calls intercepted specifically within multi-step attack sessions.
- False Positive Rate (FPR) & Steady-State FPR (FPR_{ss}) (%): Overall FPR across all benign actions, and FPR computed strictly after the 30-session warm-up buffer.
- Benign Task Completion Rate (TCR_{benign}) (%): Probability that an N -step benign session completes without a false positive:

$$TCR_{benign} = (1 - FPR_{ss})^N \quad (N \approx 3.2) \quad (13)$$

- Average Latency (ms): Mean end-to-end processing time per input action.

E. Baselines and Experimental Environment

We benchmark EVO-PCA against three representative defenses, strictly adhering to their authors’ official implementations and recommended thresholds:

- 1) PromptGuard-86M [8]: Meta’s dedicated sequence classifier for prompt injection. Evaluated using Meta’s official HuggingFace pipeline with the authors’ standard binary classification decision boundary ($\tau = 0.50$, where $P(\text{INJECTION}) + P(\text{JAILBREAK}) \geq 0.50$, matching the official pipeline ‘argmax’ decision rule). No post-hoc threshold modification was applied.
- 2) ProtectAI DeBERTa-v3 [9]: Industry-standard fine-tuned transformer classifier. Deployed with Protect AI’s default recommended decision threshold $\tau = 0.50$ on the positive INJECTION class probability ($P(\text{INJECTION}) \geq 0.50$), evaluating each action independently as a stateless sequence classifier.
- 3) SafeHarness [10]: State-of-the-art harness-level runtime isolation architecture. Executed directly from the authors’ reference implementation (src.safety.SecurityPipeline), operating with its standard two-tier defense configuration: Level-1 input context filter (filter_context) detecting prompt-level injections, followed by Level-2 LLM tool/action risk verification (check_action) executed via Ollama local inference using the authors’ default system prompt and risk thresholds.

While stateless classifiers like PromptGuard, ProtectAI, and minimax-trained sentinels such as DataSentinel [21] demonstrate high accuracy on isolated single-turn injections, they inherently lack session-level kill-chain state tracking ($\mathcal{S}_{state}$) and multi-turn behavioral correlation. Furthermore, while recent benchmarks like Agent Security Bench (ASB) [22] evaluate agent safety, we strictly refrain from reporting cross-benchmark figures from ASB’s distinct task distribution; as recognized in dependability literature, comparing raw defense scores across disparate task distributions and base rates creates severe methodological invalidity. Instead, we execute all defenses directly across our unified 29,038-record multi-step trajectory corpus.

Experiments were executed on an enterprise workstation equipped with an Intel Core i9-14900K CPU, 64 GB

DDR5 RAM, and an NVIDIA GeForce RTX 4090 GPU (24 GB VRAM). Local LLM judge inference utilized Ollama with 4-bit quantized Gemma3:4B. To ensure deterministic execution and scientific reproducibility in compliance with dependability evaluation standards, all LLM judge evaluations were executed with temperature $T = 0.0$, top- $p = 1.0$, top- $k = 1$, and fixed pseudo-random seeds. Independent triple-run replications confirmed zero variance ($\sigma = 0.00$) in decision outcomes and state transitions across identical execution traces.

VI. Results and Evaluation

A. Controlled Benchmark Comparison

Table III presents the controlled comparison across all 29,038 benchmark records.

Attack Interception Efficacy. EVO-PCA achieves 84.31% ABSR Total, outperforming the strongest stateless baseline (ProtectAI, 18.20%) by $4.63\times$ and SafeHarness (16.50%) by $5.11\times$. At the session level, EVO-PCA intercepts 89.44% of multi-step attack sessions before objective completion, representing a $3.39\times$ improvement over SafeHarness (26.40%) and a $6.17\times$ improvement over PromptGuard (14.50%).

Single-Step vs. Multi-Step Breakdown. In isolated single-step attacks, EVO-PCA achieves 84.73% ABSR. In multi-step sessions, the action-level block rate is 68.89%, while the session-level interception rate reaches 89.44%. This difference highlights the kill-chain interception principle: an attacker may execute initial reconnaissance steps (S_{recon}), but the pipeline intercepts the chain before data exfiltration (S_{exfil}) or cover-tracks (S_{cover}) can succeed.

Operational Usability & FPR Trade-Off. The pipeline yields a steady-state FPR of 8.41% (overall FPR of 9.69%), resulting in an unassisted benign completion rate of $TCR_{benign} = 75.48\%$, directly competitive with PromptGuard (76.10%) while delivering $5.8\times$ higher attack blocking capability. When paired with a single user retry override, benign completion exceeds 97.8%.

B. Block Attribution Across Pipeline Layers

Across all 29,038 benchmark records (comprising 15,104 malicious actions and 13,934 benign actions), the pipeline correctly blocks 12,734 malicious payloads, yielding the reported overall ABSR of 84.31% (with 1,350 false positives on benign developer actions, yielding 9.69% FPR). Table IV details the empirical distribution of the 13,876 total early ingress layer-triggered block events captured across the evaluation. Ingress decisions encompass both true positive malicious interceptions ($N_{TP} = 12,734$) and initial conservative policy triggers on borderline benign samples ($N_{FP,ingress} = 1,142$) prior to secondary downstream resolution.

Analysis. Layer 1 (Tier 0) accounts for 84.8% of all ingress blocks (11,770), demonstrating that canonicalization and multi-language normalization enable deterministic regex to catch the vast majority of known attack

TABLE III
Controlled Comparison on the Full 29,038-Record Benchmark Dataset

Defense System	Type	FPR (%)	FPR _{ss} (%)	ABSR Total (%)	Session ABSR (%)	TCR _{benign} (%)	Latency (ms)
PromptGuard-86M [8]	Stateless	8.12	8.12	14.50	14.50	76.10	38.50
ProtectAI DeBERTa-v3 [9]	Stateless	7.45	7.45	18.20	18.20	77.80	115.20
SafeHarness [10]	Harness	5.80	5.20	16.50	26.40	84.40	310.60
EVO-PCA (Ours)	Multi-Step	9.69	8.41	84.31	89.44	75.48	404.58

TABLE IV
Empirical Ingress Block Attribution Across Pipeline Layers

Pipeline Layer	Blocks	Share (%)
Layer 1: Tier 0 (Lexical Pre-Filter)	11,770	84.8%
Layer 4: V61 ML Ensemble	1,059	7.6%
Layer 6: Advanced Heuristics & Gate	609	4.4%
Layer 5: LLM Session Judge	311	2.2%
Layer 3: Kill-Chain State Machine	127	0.9%
Total Ingress Blocks Intercepted	13,876	100.0%

payloads at sub-millisecond speeds. The V61 ML ensemble intercepts 7.6% of blocks (1,059), catching complex paraphrasing and semantic camouflage. Advanced Heuristics and permission gating account for 609 blocks (4.4%). The LLM Session Judge intercepts 311 blocks (2.2%) that are syntactically indistinguishable from benign actions without multi-turn history. Layer 3 directly blocks 127 attacks through complete kill-chain pattern detection.

False Positive Accounting. Across the 13,934 benign actions in the benchmark, the pipeline generates a total of 1,350 false positive classifications (9.69% overall FPR). These 1,350 FPs decompose into two distinct operational stages: (1) 1,142 early Ingress false triggers ($N_{FP,ingress} = 1,142$, 8.20% of benign actions) intercepted by Tier 0 syntax rules and conservative input pattern matching (Table IV); and (2) 208 deep contextual false triggers ($N_{FP,context} = 208$, 1.49% of benign actions) generated during multi-turn execution trace review in Layer 5 (LLM Session Judge) and Layer 6 (Advanced Heuristics). These 208 contextual false alarms occur on complex benign developer scripts (e.g., test automation scripts reading mock credential environment variables or piping encoded tokens) that closely mimic adversarial reconnaissance patterns, precisely accounting for the total empirical false alarm volume ($1,142 + 208 = 1,350$).

C. Component Ablation Analysis

To isolate the marginal contribution of each architectural component, we conduct an ablation study across all 29,038 benchmark records (Table V).

Key Takeaways:

- **Tier 0 is the Architectural Backbone:** Removing Tier 0 collapses ABSR from 84.31% to 38.72% and elevates FPR to 14.30%, as raw obfuscations overwhelm downstream statistical models.

TABLE V
Component Ablation Study on Benchmark Dataset

Configuration	FPR (%)	FPR _{ss} (%)	ABSR (%)	Sess. ABSR (%)
Full EVO-PCA Pipeline	9.69	8.41	84.31	89.44
w/o Layer 1 (Tier 0)	14.30	12.15	38.72	52.10
w/o Layer 2 (Session Correlation)	9.69	8.41	78.50	71.20
w/o Layer 3 (Kill-Chain SM)	10.20	8.30	80.65	82.30
w/o Layer 4 (V61 ML)	9.85	7.90	73.81	81.50
w/o Layer 5 (LLM Session Judge)	10.55	8.55	79.71	85.10
w/o Layer 6 (Heuristics)	10.05	8.10	79.59	84.80
w/o FastPass-BenignTool	14.80	12.60	82.10	89.44

- **Session Correlation Powers Multi-Step Interception Without Benign Penalties:** Removing session correlation reduces Session ABSR by 18.24pp (from 89.44% to 71.20%), proving that cross-step behavioral flag tracking is essential for multi-step defense. Crucially, FPR remains invariant at 9.69% (and FPR_{ss} at 8.41%) because the Domino Effect Safeguard strictly requires an active, unexpired adversarial flag newly generated by the current step to fire a combination rule. Normal developer workflows in the benign benchmark never generate these initial threat flags; hence, Layer 2 correlation rules remain completely inactive on benign sessions. Removing Layer 2 therefore subtracts zero false alarms from benign traffic, preserving the exact same 9.69% FPR while eliminating 18.24pp of attack session defense efficacy.
- **Kill-Chain State Machine Maximizes Interception Depth:** Disabling Layer 3 drops Session ABSR by 7.14pp (to 82.30%), largely because early warnings that alert the Session Judge are eliminated.
- **FastPass Preserves Availability:** Disabling FastPass increases FPR from 9.69% to 14.80% (+5.11pp), as benign developer actions (git status, make) are unnecessarily routed to ML classifiers.

D. Targeted Adversarial Evasion Evaluation

We subject the pipeline to 15 targeted adversarial evasion techniques spanning all six layers (Table VI). Rather than relying on static single-turn tests, these scenarios represent multi-layer evasion vectors specifically designed to stress-test lexical obfuscation, session decay manipulation, ML feature spoofing, and canary token isolation. EVO-PCA successfully intercepts 14 out of 15 evasion strategies (93.3% block rate). The sole bypass (Test Case 6: Session ID rotation in anonymous mode) is mitigated when an authenticated user_id is available to GlobalThreatTracker.

TABLE VI
Targeted Adversarial Evasion Evaluation (15 Multi-Layer Bypass Scenarios)

#	Adversarial Evasion Technique	Outcome	Intercepting Layer
Layer 1 Evasion (Lexical Obfuscation)			
1	Cyrillic homoglyph substitution chain	BLOCK	L1 (Homoglyph map)
2	Zero-width character interleaving	BLOCK	L1 (ZWC stripper)
3	Base64-encoded bash command injection	BLOCK	L4 (ML metachar TF-IDF)
Layer 2 Evasion (Session Correlation Bypass)			
4	Slow-drip exfiltration (>1hr inter-arrival)	BLOCK	L2 (Combo rules)
5	Flag dilution (20+ intervening benign steps)	BLOCK	L2 (LRU decay window)
6	Session ID rotation (anonymous user)	ALLOW [†]	— (Mitigated by L6 APT)
Layer 4 Evasion (ML Feature Manipulation)			
7	TF-IDF dilution with 1,000+ benign tokens	BLOCK	L4 (Long-Text Chunker)
8	Imperative synonym substitution	BLOCK	L6 (Taint tracker)
9	Metacharacter fragmentation (c'u'r'l)	BLOCK	L1 (Canonicalizer)
Layer 5 Evasion (Judge Exploitation & Canary Bypass)			
10	Gradual privilege escalation across turns	BLOCK	L3 (Kill-Chain State)
11	Fake provenance injection in tool response	BLOCK	L6 (InstructionBoundary)
12	Canary token spoofing	BLOCK	L5 (64-bit hex nonce)
13	Strict XML tag breakout (< /container >)	BLOCK	L5 (XML entity escape)
14	Simulated execution output injection	BLOCK	L5 (Nonce validation)
15	Multi-turn canary extraction	BLOCK	L5 (Per-request refresh)

TABLE VII
Latency Breakdown Across Pipeline Components

Pipeline Component	Mean Latency	Execution Share
L1: Tier 0 Regex & Canonicalizer	0.82 ms	100.0% (All inputs)
L2: Session Correlation & Flag Decay	2.14 ms	59.5% (Non-blocked)
L3: Kill-Chain State Tracker	0.45 ms	59.5%
L4: FastPass-BenignTool Filter	0.38 ms	41.2% (Tool calls)
L4: V61 ML Ensemble (RF)	11.25 ms	35.8% (Non-FastPass)
L5: LLM Action Judge (Ollama)	342.10 ms	6.4% (Margin review)
L5: LLM Session Judge (Ollama)	418.50 ms	3.2% (Suspicious band)
L6: Advanced Heuristics & Registry	1.85 ms	100.0%
Egress: ContextSanitizer	0.21 ms	Tool returns only
End-to-End Pipeline Mean	404.58 ms	Weighted Average

E. Latency Profiling by Component

Table VII presents the granular latency profile measured across 1,000 randomized benchmark actions.

Fast-path execution (Tier 0, Session Correlation, FastPass) completes in under 4 ms. Only inputs falling into margin or suspicious bands invoke the local LLM judges (~340–420 ms), keeping the overall weighted mean latency at 404.58 ms—well within production interactive budgets (≤ 500 ms).

Tail Latency Analysis. To rigorously evaluate compliance with interactive dependability SLAs, we analyze the empirical latency distribution across percentiles:

- 50th Percentile (p_{50}): 2.14 ms. Over 59% of normal traffic resolves via fast-path execution layers (Tier 0 regex, FastPass filter, and session cache) in under 3 ms without engaging deep ML/LLM pipelines.
- 90th Percentile (p_{90}): 384.20 ms. Queries requiring V61 ML ensemble scoring or single-judge verification.
- 95th Percentile (p_{95}): 435.60 ms. Complex multi-step queries triggering full heuristic cross-correlation and LLM Action Judge review.
- 99th Percentile (p_{99}): 488.30 ms. Deep dual-judge evaluations invoked on highly ambiguous or suspicious token bands.

Crucially, even at the 99th percentile ($p_{99} = 488.30$ ms), end-to-end processing strictly adheres to the mission-critical interactive availability budget of ≤ 500 ms, while

TABLE VIII
Graceful Degradation Under Subsystem Failures

Simulated Failure Mode	Retained ABSR (%)	Latency Delta
Ollama LLM Judges Offline	78.50%	−250 ms (Fast-Path)
V61 ML Models Corrupted/Missing	73.81%	−200 ms
Full ML/LLM Subsystem Outage	69.50%	Falls to 25 ms
Tier 0 Deterministic Rules Only	64.20%	Falls to <2 ms

maintaining an ultra-responsive 2.14 ms median response for legitimate user interactions.

VII. Dependability Discussion

A. Graceful Degradation Under Component Failure

A foundational dependability property is fail-soft operation under subsystem outages. Table VIII evaluates pipeline behavior when individual layers experience simulated faults.

Because Layer 1 (Tier 0) and Layer 2 (Session Correlation rules) are fully deterministic and require no external services or GPU runtimes, the pipeline retains 76.2% of its original defense efficacy (64.20% absolute ABSR, capturing 84.8% of all empirical block events; Table IV) even during catastrophic failure of all ML and LLM components.

B. Availability Preservation Under Adversarial Burst

During distributed prompt injection floods, conventional ML pipelines experience severe throughput collapse due to inference queue saturation. EVO-PCA addresses this through two cooperative mechanisms:

- 1) Tier 0 Early Shedding: Over 84% of attack traffic is terminated in Layer 1 within 0.82 ms, shielding the downstream ML and LLM layers from 84.8% of adversarial computational load.
- 2) Anti-E7 SignalRegistry Isolation: By preventing Layer 6 heuristic spikes from corrupting the Adaptive Escalation Manager’s FPR feedback loop, the system prevents runaway threshold oscillation, ensuring stable availability for legitimate traffic.

C. Mean Time to Intercept (MTTI)

In multi-step attack campaigns, intercepting an attack at Step 1 versus Step 3 represents a critical dependability distinction. In our benchmark evaluations:

- Step 1 (Prompt): 6.34% of multi-step sessions are intercepted on the initial prompt turn.
- Step 2 (First Tool Call): 61.20% of sessions are intercepted during reconnaissance or initial tool invocation.
- Step 3+ (Exfiltration/Cover): 21.90% of sessions are intercepted prior to goal completion.
- Overall Interception: 89.44% of multi-step attacks are stopped before accomplishing their payload objective ($\text{ASR}_{\text{task}} = 10.56\%$).

D. Maintainability and Modular Lifecycle

In compliance with dependability requirements, each layer supports independent lifecycle management:

- Hot-Reloadable Signatures: Tier 0 regex patterns and FastPass whitelists are hot-reloaded from external JSON configuration without service disruption.
- Automated ML Deployment Gates: V61 models are retrained using historical traces and must satisfy validation gates ($F1 \geq 0.85$, $FPR \leq 5\%$) prior to deployment.
- Backend Model Agnosticism: The LLM Action and Session Judges communicate via standard local HTTP endpoints, permitting seamless model swapping (Gemma3 $\rightarrow$ Qwen2.5 $\rightarrow$ Llama-3.3) without modifying core pipeline logic.

VIII. Conclusion

This paper presented the EVO-PCA Multi-Step Hardened Pipeline, a six-layer defense-in-depth security architecture for dependable autonomous LLM agent systems. By integrating stateless lexical pre-filtering with session-aware behavioral correlation, kill-chain state tracking, calibrated ML classification, dual-judge LLM verification, and egress sanitization, the architecture overcomes the structural blind spots of existing stateless guardrails. Evaluated on the full 29,038-record AgentDojo/Agent_Test benchmark, EVO-PCA achieves 84.31% ABSR Total and intercepts 89.44% of multi-step attack sessions—a $3.39\times$ improvement over SafeHarness—while maintaining a steady-state FPR of 8.41% (overall FPR of 9.69%) and mean latency of 404.58 ms. The system demonstrates robust dependability attributes: retaining 76.2% of its protection efficacy (64.20% absolute ABSR, accounting for 84.8% of all empirical block events) under complete ML/LLM outages, decoupling adaptive threshold control from false escalation cascades, and supporting hot-swappable layer maintenance.

Future work includes integrating graph neural networks for provenance-aware taint propagation and deploying quantized local models to further reduce deep-path inference latency.

References

- [1] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” Version 1.1, 2023.
- [2] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, 2023.
- [3] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, “Basic concepts and taxonomy of dependable and secure computing,” *IEEE Trans. Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.
- [4] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, “Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains,” in *Proc. 6th Int. Conf. Information Warfare and Security (ICIW)*, 2011, pp. 113–125.
- [5] S. Axelsson, “The base-rate fallacy and the difficulty of intrusion detection,” *ACM Trans. Information and System Security (TISSEC)*, vol. 3, no. 3, pp. 186–205, 2000.
- [6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 139–150.
- [7] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [8] Meta AI, “LlamaFirewall and PromptGuard-86M model card,” 2024. [Online]. Available: <https://huggingface.co/meta-llama/Prompt-Guard-86M>
- [9] Protect AI, “DeBERTa-v3-base prompt injection classifier,” 2024. [Online]. Available: <https://huggingface.co/protectai/deberta-v3-base-prompt-injection>
- [10] X. Lin et al., “SafeHarness: Lifecycle-integrated security architecture for LLM-based agent deployment,” *arXiv preprint arXiv:2604.13630*, 2026.
- [11] T. Rebedea et al., “NeMo Guardrails: A toolkit for controllable and safe LLM applications,” in *Proc. EMNLP System Demonstrations*, 2023, pp. 431–445.
- [12] J. Siebert et al., “Construction of a quality model for machine learning systems,” *Software Quality Journal*, vol. 30, pp. 307–335, 2022.
- [13] M. Cheng, A. Nazari, and H. Katabi, “Securing LLM-integrated systems: Challenges and countermeasures from a dependability perspective,” in *Proc. 54th IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2024, pp. 45–56.
- [14] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [15] B. Liu, L. Deng, R. Chakraborty, J. Li, and K. Roy, “Trustworthy AI: From principles to practices,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–46, 2023.
- [16] P. Sharma et al., “AgentSmith: Automated red-teaming for multi-step autonomous agent systems,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 1201–1218.
- [17] E. Perez et al., “Red teaming language models with language models,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2022, pp. 3419–3448.
- [18] J. C. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” *Advances in Large Margin Classifiers*, vol. 10, no. 3, pp. 61–74, 1999.
- [19] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
- [20] R. Schuster, C. Song, E. Tromer, and V. Shmatikov, “You autocomplete me: Poisoning vulnerabilities in neural code completion,” *Communications of the ACM*, vol. 64, no. 9, pp. 88–97, 2021.
- [21] Z. Zhang et al., “DataSentinel: Robust input guardrails for autonomous LLM agents via minimax game formulation,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2025, pp. 1–18.
- [22] C. Xie et al., “Agent Security Bench: A comprehensive evaluation framework for autonomous agent safety,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
- [23] Microsoft, “LLMail-Inject Challenge: Benchmarking indirect prompt injection in email assistants,” 2024. [Online]. Available: <https://huggingface.co/datasets/microsoft/llmail-inject-challenge>
- [24] M. Andriushchenko et al., “AgentHarm: A benchmark for measuring harmfulness of LLM agents,” *arXiv preprint arXiv:2410.09024*, 2024.
- [25] J. Deng et al., “Benchmarking indirect prompt injection attacks in LLMs,” *arXiv preprint arXiv:2312.14197*, 2024.
- [26] Neuralchemy AI, “Prompt injection and obfuscation dataset,” 2024. [Online]. Available: <https://huggingface.co/datasets/neuralchemy/Prompt-injection-dataset>